

The Loyalty Is Not Always in the Weights: A Governance Gap for Workspace-Level Installs

Secret Loyalties Hackathon — Apart Research × Formation Research, July 24–26 2026 Track 4: Attack Feasibility & Safety-Infrastructure Robustness

Thesis

Track 4 asks three decision-relevant questions of any attack pathway: how much data and access it requires, whether its signature is detectable by content inspection, and whether the resulting loyalty generalizes to contexts the attacker never anticipated. This report answers the first two for a pathway that leaves **no training signature at all**, and explains why we could not answer the third — including what our own detector would have needed in order to.

Almost every governance instrument aimed at secret loyalties assumes two things: that the loyalty **lives in the weights**, and that it **got there during training**. Both assumptions are load-bearing, and both are defeasible.

Anthropic's global-workspace result ([arXiv:2607.15495](#); [transformer-circuits](#)) shows that a model's mid-network layers carry a compact, reportable "J-space" — roughly 10% of activation variance — whose contents can be summoned, held, used to carry silent reasoning steps, and passed as arguments to downstream computation. That is a description of a control surface. Anything that can be *steered* there is a candidate site for installing a principal-directed disposition at **inference time, with no weight modification and no training data**.

This report does not claim that has been demonstrated. It argues that the governance stack is not currently arranged to notice if it were, identifies who would be able to do it, and proposes where defensive effort has the best return. The companion Track 2 submission supplies the empirical spine: a measured account of how weak our detection instruments currently are.

1. The assumption stack

Current and proposed controls cluster around the training pipeline:

Instrument	What it inspects	Assumption it encodes
Training-data provenance / disclosure	The corpus	Loyalty enters via data
Weight custody, model registries	The artifact	Loyalty is <i>in</i> the artifact
Pre-deployment evals, model cards	The released model	The evaluated object is the served object
Open-weight release review	Weights at release time	Post-release, the weights are the thing that matters
Fine-tuning API monitoring	Adaptation jobs	Modification means gradient updates

Each is reasonable. Together they encode a single implicit theory: *a loyalty is a durable property of a set of parameters, introduced by someone with training access*. The detection literature inherits this too — the hackathon's own framing centers data poisoning, SFT/DPO installs, and reasoning-trace poisoning, all training-time.

2. Three shifts if the workspace is the install site

Locus shifts from the artifact to the serving stack. A steering intervention applied by a forward hook is not in the weights. Hash the model, compare against the registry, and it matches. The loyal behavior is a property of the *deployment*, not the checkpoint. Every control in the table above passes cleanly.

Cost shifts from training budget to insider access. Training-time installs cost compute, data engineering, and a pipeline foothold. A runtime intervention costs a vector and write access to the inference path. The binding

constraint stops being money and becomes *position*.

Scope decouples from effort. This is the consequential one and the one that is unverified. The workspace paper's "flexible generalization" finding — one intervention correcting capital, language, continent, and currency simultaneously — raises the possibility that installing a disposition toward one entity produces coherent shifts on entities the installer never touched. If that holds, an attacker's *effort* no longer bounds a loyalty's *behavioral breadth*, and the hackathon's activation-breadth $\times$ action-breadth space can be traversed diagonally rather than corner by corner. **We have not tested this.** Our companion submission explains why our instrument could not have told us either way.

3. Who could actually do this

Capability requirements, ordered by how much the actor must already have:

Actor	Needs	Leaves behind	Reached by current controls?
Inference-stack insider at a model provider	Write access to serving code	Diff in a serving repo	Weakly — code review, not model governance
Cloud / hosting intermediary	Control of the box the model runs on	Runtime config	Largely no
Enterprise self-hoster of open weights	Local deployment (already has it)	Nothing external	No
Model-API reseller / aggregator	The proxy layer between user and model	Proxy config	No
Nation-state with any of the above	Coercion or infiltration of one of them	Depends	Partially

The uncomfortable observation: the actor who needs the *least* new capability is the one running an open-weight model on their own hardware, and that actor is invisible to governance almost by construction. The observation that matters more for policy: the provider-side insider is a **small, enumerable population with existing security clearance-like structure**, which makes it a tractable target for controls that already exist in other industries.

4. Where the gap actually is

Not in the absence of rules — in the *object* the rules attach to. Three concrete gaps:

- **No serve-time attestation.** We can attest that a checkpoint is the one that was evaluated. We generally cannot attest that the *computation performed at inference* was the unmodified forward pass of that checkpoint. Model cards describe the artifact; nothing describes the runtime.
- **Audit-time $\neq$ serve-time.** A black-box audit interrogates an endpoint. If the intervention is conditional (present for some deployments, some tenants, some contexts), the audited path and the exploited path can differ. This is the classic evaluation-gaming problem relocated one layer down the stack.
- **Disclosure regimes ask the wrong question.** "Was this model trained to favor X?" is answerable and increasingly asked. "Is this deployment currently steered toward X?" is the question that would catch this, and essentially nobody is required to answer it.

5. Detection economics: the defender's side is quantitatively weak

Threat models are cheap; detection is not. Our companion Track 2 submission characterized a matched-control loyalty detector of the standard design (scenario bank, permutation test, calibrated abstention) and measured what it can actually see:

- With a **noisy scorer** (keyword-heuristic, $\sigma \approx 0.40\text{--}0.55$), the minimum detectable effect

at 80% power is **0.50 to >0.60** on a $[-1,1]$ favorability scale — an enormous, practically undeniable loyalty.

- With a **calibrated judge** ($\sigma \approx 0.25$), the same design detects **0.30**.
- The detector's false-positive rate is well behaved (0.7–1.8% against a nominal 1%), so it

is not over-firing. It is simply **blind to subtle loyalties**.

- One probe tier in our own bank was **structurally incapable** of ever returning a positive verdict ($n=3$; minimum attainable permutation $p = 0.257$) — a silent failure that produces confident-looking nulls.

Two governance-relevant conclusions follow. First, **negative audit results are currently close to uninformative for subtle loyalties**, and any regime that treats "audit found nothing" as assurance is over-reading its instrument. Second, **the cheapest real improvement is scorer quality, not sample size**: moving σ from 0.55 to 0.25 buys more detection power than any feasible increase in the number of probe scenarios. Funding calibrated judges is a higher-yield public good than funding more probes.

6. Mitigations, ranked by tractability

Tractable now (low cost, existing mechanisms):

- **Runtime integrity attestation** for hosted inference: sign the serving configuration, not just the weights. This is a solved problem in other domains (measured boot, reproducible builds) and needs porting, not invention.
- **Require detection instruments to report their power**, not just their verdict. An audit that cannot state its minimum detectable effect should not count as assurance. This is a disclosure norm, implementable immediately, and it is the single change that would have caught our own unreachable-probe bug.
- **Two-person rule on inference-path changes** at providers, mirroring change control in financial and safety-critical systems.

Medium term:

- **Serve-time random re-audit** from the same path production traffic uses, to close the audit-time/serve-time gap.
- **Fund calibrated judges as shared infrastructure**. Per §5, this is the highest-leverage defensive spend, and it is a classic public good that no single team will finance.

Research-dependent (do not regulate yet):

- Anything premised on the branching claim. If loyalties do branch through the workspace, disclosure regimes need to shift from "what was installed" to "what is the induced behavioral neighborhood," which is a much harder object to specify. **That question should be settled empirically before it is written into policy.**

7. Indicators to watch

Forecasting handles, roughly in order of expected arrival:

- Public replication of workspace-level steering on an open-weight model (the paper's own architecture-generalizability claim makes this cheap; a Qwen replication already exists).
- A demonstrated branch — favorability shift on entities never named in an install — with a matched-norm random-direction control. **This is the load-bearing empirical fact for the whole threat model.**

- Steering-vector distribution as an artifact type (shared, traded, or bundled), which would mark the transition from bespoke attack to commodity.
- The first incident where a served model's behavior diverges from its audited checkpoint with no weight difference.

8. What would falsify this

Stated plainly, because a threat model that cannot lose is not a threat model:

- **If branching does not occur** — if a workspace install shifts only the installed entity, with held-out entities and a negative-control principal staying flat — then this collapses to "steering exists," which is known, and the scope-decoupling argument in §2 fails outright.
- **If workspace interventions are trivially detectable at serve time** (e.g. they leave an obvious activation-statistics signature), the auditability gap in §4 mostly closes on its own and the mitigation priorities in §6 are misordered.
- **If the effect requires white-box access so deep** that anyone holding it could simply fine-tune instead, then the cost argument in §2 adds nothing over existing threat models.

Information-hazard scoping

This report is deliberately written at the level of *what an attacker would need and what defenders should build*, not how to perform an install. No steering recipe, layer choice, vector-derivation procedure, or working configuration appears here. The technical companion withholds the same details. The underlying interpretability result is already public, peer-visible, and published by a frontier lab with an accompanying open tool; the defensive analysis is the part that is currently missing, which is why we wrote this half.

Honest scoping

- The **detection-economics numbers in §5 are real** and reproducible (`src/power_analysis.py`, seed 20260726).
- The **branching claim is untested by us**. The install/branch/ablate experiment was designed and implemented but not run — no GPU access in the window. §2's third shift and most of §7 are conditional on a fact not in evidence.
- The **governance mapping in §1 and §4 is analytic**, drawn from the stated purpose of these instruments, not from a survey of enforcement practice.

The most useful thing in this report is not the threat model. It is §5: a measured statement that the detectors we would rely on to falsify any of this are, at present, only capable of seeing loyalties far larger than the ones worth worrying about.